

长会话开始遗漏约束， 应该怎样调整上下文和工具加载？

先改信息供给：查清出错那一步模型实际收到了什么，把约束、进度和文件版本记在每一步都携带的任务状态里；改提示词和扩大窗口只作对照。中途新增工具的额外费用取决于定义加载的位置，与工具个数没有固定的倍数关系。

到了第 40 轮，模型开始忘记要求、反复读同一份文件。先看出错那一步实际收到的请求，常见的有四种情况，各自指向不同的修改。

约束的状态	原因	修改位置
请求中没有该约束	未保存，或恢复任务时未读回	保存与恢复流程
约束在压缩后消失	摘要删去了约束	压缩规则：关键约束、数字和禁止事项压缩后仍能查回原文
约束仍在，模型未遵循	与其他指令冲突、位置不当或任务过难	提示词、约束位置与 context window：只在这种情形下比较
约束已被后续指令修改	不属于遗忘	任务状态：记录变更及生效时间

前两种情况，约束在送达模型之前已经缺失，提示词写得再好也无法补回；扩大窗口最多推迟压缩，删去约束的那条压缩规则仍然有效。

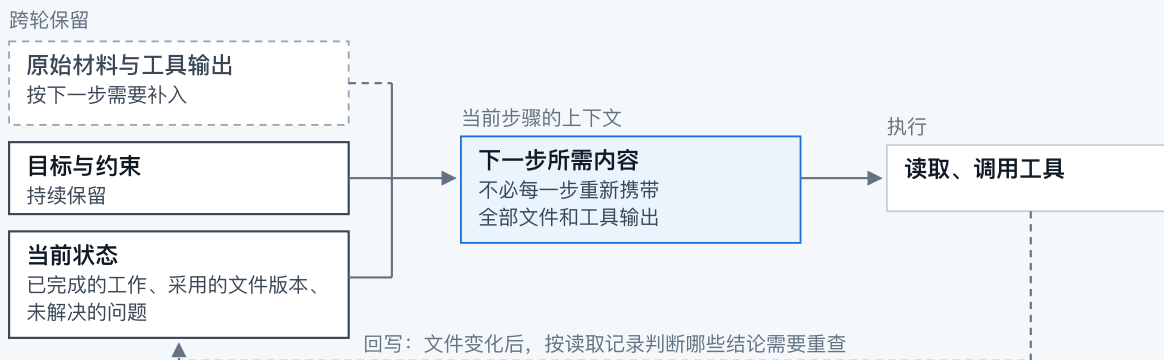
1.1 怎样保留任务状态，又避免每轮重读全部材料？

目标、约束和进度每一步都携带，文件和工具输出用到时再读取。文件修改后，只重查依赖旧版本的那些结论。

长任务进行到一半，最终要交付什么、上一步做到了哪里，这两点都要留在当前上下文里。这样模型不必靠重翻文件来找回进度；文件原文等下一步确实用到时再读取。

图 1.1

保留目标和进度，按需读材料，再把结果写回进度记录



进度不能只写“合同已检查”，要写清查的是哪一版、结论是什么、还剩哪些问题。文件一更新，顺着这份读取记录就能找到该重查的结论；任务中断了，也能从这里接着做，不会把“读过”误当成“现在仍然有效”。

有了读取记录，也就分得清哪些重读是必要的。文件改过、要核对原文、上次只读了一部分，这些都该再读；同一版本、同样目的、读完也没有新信息的重读，才需要减少。至于权限和客户隔离，不依赖这份记录，也不依赖模型的记忆，每次调用工具都重新检查。

1.2 第 40 轮新增三个工具，会让旧内容重新计费吗？

不一定。新定义如果改动了已缓存的前缀，受影响的旧内容需要重新处理；如果只是追加在会话后面，旧缓存仍然可能命中。

新增三个工具，费用要分两笔计算：一笔是新定义、发现调用和工具执行本身的花费；另一笔是原本能命中缓存的旧内容，是否因此产生额外费用。第二笔取决于定义放在哪里，仅凭“第 40 轮新增了工具”，推不出整段上下文都要重算。

图 1.2

同一批旧内容在三种加载路径下的差额

加载路径	对旧内容的影响	旧内容的差额
修改已缓存前缀	受影响的旧内容重新处理	$150,000 \times (1 - 0.1) \div 1,000,000 = 0.135$ 元
在会话后部追加定义	旧前缀可能继续命中	旧缓存命中时为 0
只更新可搜索目录	完整 schema 可能尚未载入	核对目录摘要和实际加载记录后计算，不能直接记为 0

比较前提：模型和其他配置不变，原缓存仍有效。假设受影响的旧内容为 150,000 token，重新处理为每百万 token 1 元，读取缓存为每百万 token 0.1 元。

旧内容如果重新处理，按接口实际适用的普通输入或缓存创建计费，同一批 token 只算一次。旧前缀继续命中，只说明这部分没有多付；新定义、发现调用、执行和返回的内容仍要另算。

按需发现省下的是每次请求都携带整张工具目录的开销，代价是发现本身可能多一次调用。少数几个工具每步都要用，预先加载更简单；目录很大、每次只用其中几项，按需发现才更可能划算。运行时本身支持 tool search 或 deferred loading，就直接使用，不必再自行封装一层。

“中途加入工具”包含四个环节，前一个环节完成，不等于后一个也已完成：

环节	回答的问题	不能据此推出的结论
发现	目录里有哪些相关工具	工具已获准执行
定义载入	schema 何时、在什么位置进入模型上下文	工具服务已经接通
服务接入或热更新	会话不中断时，能否接入、更新或移除工具实现	缓存不受影响，或进行中的调用可以换到新版本
调用授权	现在能否对这个对象执行这个动作	加载或批准一次后可以长期执行

追加定义不影响旧缓存，撤下或改动已载入的定义则不同：改动位置之后的缓存都会失效。客户端收到目录变更通知，也不代表定义已经载入、服务已经接通。要确认本次采用哪种加载路径、前面的估算是否成立，需要沿请求轨迹查看定义在哪一轮、什么位置出现，再用 usage 核对缓存命中。

1.3 怎样证明这些调整改善了长任务？

用同一组长任务做对照，每次只改变工具加载、上下文组织或窗口大小中的一项，再比较结果和总开销。

账单降了，不等于模型不再漏掉约束。按需发现可能多一次调用，选错工具还会引出重试；如果同时重组上下文、扩大窗口，就算任务做得更好，也无法区分改善来自哪一项。

待验证的改动	保持不变的条件	比较指标
预加载与按需发现	任务、模型、材料、权限、上下文组织和窗口	工具选择与执行结果、重试、总费用和 p95
原上下文与分层组织	同组长任务、模型、权限、工具加载和窗口	约束遗漏、失效文件误用、不必要重读、恢复成功率、成本和 p95
原窗口与扩大窗口	同组长任务、模型、权限、工具加载和上下文组织	相同的任务质量与开销指标

检查点不只放在第 40 轮：遗漏不一定总在那里出现，第 20、40、60 轮都要看。上下文变短了却缺失必要事实，不算改好；加载便宜了却让任务更难恢复，这笔代价也要算进去。最后保留的，应当是任务照样完成、总开销确实下降的改动。

来源

01	Anthropic — Effective context engineering for AI agents https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents 按需检索与结构化笔记供给	Anthropic · 官方工程文章
02	Claude Code — How Claude Code uses prompt caching https://code.claude.com/docs/en/prompt-caching 缓存按前缀匹配，改动即失效	Anthropic (Claude Code) · 官方文档
03	OpenAI — Tool search https://developers.openai.com/api/docs/guides/tools-tool-search 按需发现的工具追加在末尾	OpenAI · 官方文档
04	Anthropic — Tool search tool https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool 延迟加载的定义不进入初始上下文	Anthropic · 官方文档
05	OpenAI — Prompt caching https://developers.openai.com/api/docs/guides/prompt-caching 追加工具可保留旧前缀缓存	OpenAI · 官方文档
06	Model Context Protocol — The 2026-07-28 Specification https://blog.modelcontextprotocol.io/posts/2026-07-28/ 目录变更通知不等于已接通	Model Context Protocol · 官方工程文章
07	OpenAI — Evaluation best practices https://developers.openai.com/api/docs/guides/evaluation-best-practices 持续评估捕捉配置变化	OpenAI · 官方文档

怎样在能力、等待时间和成本之间选择？

不给整项任务固定一个 reasoning effort，而是按每一步的难度和出错代价来选；衡量的是多久出现有用结果，以及完成工作的总费用。

提高 reasoning effort，有时能省下后面的重试和返工，有时只是多等了时间。所以“推理更强”和“调用更贵”要放在一起算，一直算到任务结束，看多花的时间和费用换来了什么。

2.1 什么时候值得提高 reasoning effort？

难生成、容易返工的步骤，值得试更高的 effort；有确定算法的步骤交给代码。缺的是事实，就先找材料，推理再久也推不出证据。

答案好检查，不代表好做出来。程序是否通过测试，运行一次就能知道；找到正确的修复，却可能要试好几轮。低 effort 如果总是失败，还要反复检查、修改，一开始就提高 effort 反而可能更省。



边界清楚、检查成本可控的步骤，可以先用较低 effort 运行一次，再和升级后的正确率、重试次数、总成本比较。任务进行中也可以调整：抽取信息时发现材料互相矛盾，就重新考虑要不要升级；复杂规划里夹着的机械计算，仍然交给代码。选模型也是同样的道理，看它在相似任务、相同工具环境和质量要求下的实际表现。各家的 effort 档位对应的计算量并不一样，只能拿同一批任务实测比较。

工具选错，有时在排序之前就已经注定：正确的工具根本没被检索出来，排序再准也选不到它。所以评估要从候选工具是否齐全查起，一直查到选中的工具能否完成工作；只看请求和工具描述是否相似，是不够的。

有外部副作用的步骤另有规则。发邮件、覆盖文件、改线上配置之前，effort 设得再高、模型想得再清楚，也不等于获准执行；是否放行，由应用在执行前检查授权来决定。

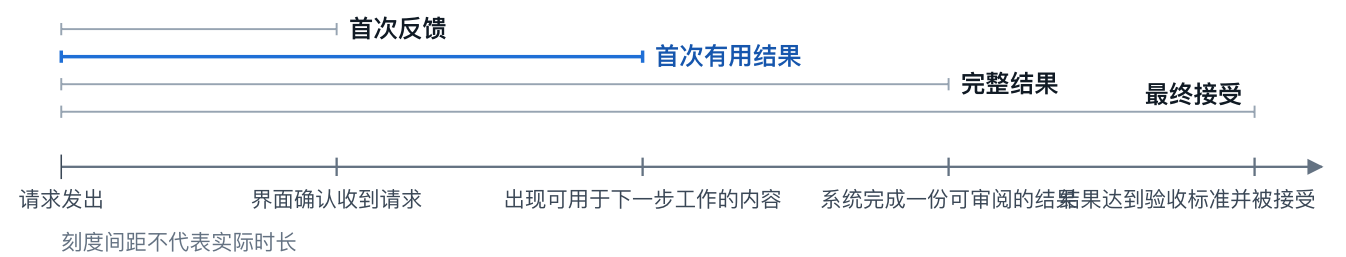
按步骤选 effort，并不是要在同一条长会话里来回切换模型或 effort。有些实现把这两项算进缓存键，切换后旧内容需要重新处理。可以把不同阶段交给职责固定、上下文较短的调用，编排增加的开销也一并计入。

2.2 怎样判断用户的等待是否合理？

从请求发出开始计时，重点看多久出现有用结果；首次反馈、完整结果和最终接受也分别记录。

界面一秒内出现“正在处理”，只说明系统收到了请求。之后如果很久都没有能用的内容，用户其实仍在等待。衡量等待，关键不在模型何时开始输出，而在它何时给出能用于下一步工作的结果。

图 2.2
四项时间都从同一次请求发出算起



初版先按下表设定目标，上线后按任务类型统计实际延迟和用户行为，再决定怎样调整。

初始目标或触发条件	产品行为
请求发出后 1 秒内	确认收到请求
简单任务首次有用结果的p95 不超过 3 秒	以可用内容出现计时，不能由打字动画代替
预计任务超过 10 秒	显示已经完成的步骤，允许取消、离开和返回

长任务可以先交出有用的一部分，让后续工作先开始。但整份结果如果还要审阅、修改，这段时间也要计入最终接受；第一段来得早，不能用来掩盖后面漫长的返工。

2.3 成本应该算到一次调用，还是一次完成的任务？

把模型、工具、重试、人工审阅和返工的费用加起来，再除以最终被接受的结果数；一份也没被接受时，单独报告失败的费用。

一次调用便宜，可能只是把更多工作留给了下一次调用或人工。所以比较人工完成和 AI 辅助，要把同一项工作从头算到尾，不能拿模型单价去比人工工时。

图 2.3
人工与 AI 辅助完成同一项工作的费用（假设演算）

人工完成	人工 12 分钟 60	60 元
AI 辅助	人工 5 分钟 25 ← 模型与工具费 2	27 元
未计入	新增错误带来的返工或其他损失	—

时间价值每小时 300 元；同一尺度 0–60 元

假设演算里省下的 33 元还不是净收益，新增错误带来的返工或损失还要扣除。提高 effort 或换模型也这样算：错误概率降低多少，乘以每次出错的损失，超过新增的模型费、等待和验证成本，升级才划算。这笔经济账在满足安全要求之后才算，省下的钱抵消不了越权。

2.4 不展示原始思考，怎样让工作过程可信？

把用了哪些材料、改了什么、执行结果如何展示出来，让每个判断和动作都能核对。

检查文件时，显示读的是哪一版、发现了哪些冲突、还缺哪些事实。这样用户知道工作为什么停在这里，也能指出遗漏。准备发送或修改文件时，再把具体内容、目标对象和影响范围列出来，用户确认的就是马上要发生的那个动作。

执行后给回执，并核对结果是否符合预期，“准备做”和“已经做成”才分得开。这些记录还能用于中断恢复和事后审阅，一段读来流畅的思考过程代替不了它们。

结果确认不了，界面就照实显示。例如进程已经结束、输出也已保存，服务却在发布结果前中断：恢复以后可以展示已有输出，但执行状态仍标“未确认”，也不提供重试按钮。按钮与真实状态一致，比写一段解释更能帮助用户判断下一步。

来源

01	OpenAI — Reasoning models https://developers.openai.com/api/docs/guides/reasoning effort是速度、成本与质量的取舍	OpenAI · 官方文档
02	jev-harness-lab — Technical evaluation report https://github.com/Aitejiu/jev-harness-lab/blob/main/docs/REPORT.md 候选召回限制排序的上限	Aitejiu · 研究代码仓库
03	Anthropic — Steering thinking and cost https://platform.claude.com/docs/en/build-with-claude/thinking-steering-and-cost 节点成本需累加到任务级	Anthropic · 官方文档

04	OpenAI — Latency optimization https://developers.openai.com/api/docs/guides/latency-optimization 减少请求并改用确定性逻辑	OpenAI · 官方文档
05	Claude Code — How Claude Code uses prompt caching https://code.claude.com/docs/en/prompt-caching 切换 effort 可能导致缓存重算	Anthropic (Claude Code) · 官方文档
06	OpenAI — Evaluation best practices https://developers.openai.com/api/docs/guides/evaluation-best-practices 评估须同时衡量质量、延迟与成本	OpenAI · 官方文档

哪些动作可以自动执行，失败后又怎样恢复？

已获授权的动作可以继续执行；发送、披露或修改数据以后，如果不知道是否成功，就先查结果，不能直接重做。

同样是改文件，云端沙箱可能连着生产账号，本地桌面上打开的也可能只是工作副本。要不要确认，看的是这次会访问什么、改什么、把数据发到哪里，而不是程序运行在云端还是本地。

3.1 什么时候需要重新取得确认？

账号、对象、内容或后果超出了已有授权，就在执行前确认；还在授权范围内的，不必重复询问。

允许读一封邮件，不等于允许把它发出去，哪怕用的是同一个账号、同一个工具。所以要确认的是这一次具体要做的事，而不是笼统地问“能不能用这个工具”。

图 3.1
按具体动作的后果决定是否需要确认

授权范围内：继续执行

执行前：确认授权覆盖具体后果

读取

发送

整理

披露

起草

覆盖关键文件

改变生产状态

批准发送时，要定下收件人、正文版本和有效期；修改资源时，还要定下改动范围。正文、收件人或目标一变，就要重新看原来的批准是否仍然适用。网页或邮件里要求上传文件，那只是读到了一条外部指令，不能当成用户已经同意。

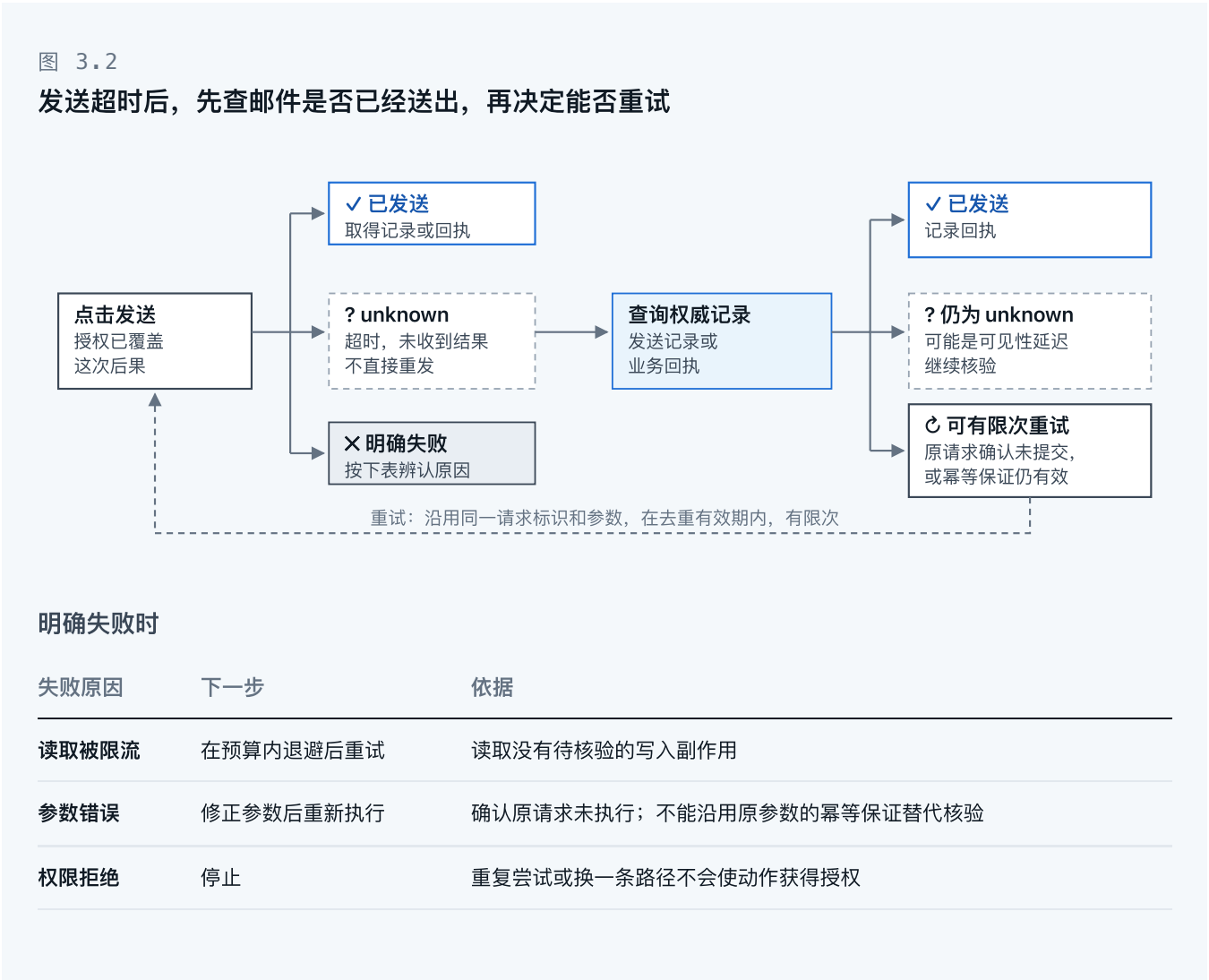
确认必须在数据离开之前完成。敏感内容一旦填入联网表单，就可能已经上传；等到点击提交时再询问，这次披露已无法阻止。

3.2 Agent 失败后，什么时候可以自动重试？

只有确认原请求已经终止且没有提交，或者接口仍能保证同一请求不会重复产生副作用，才可以有

限次重试。遇到权限拒绝，必须停止。

点击发送后超时，邮件可能已经发出，只是客户端没有收到回执。这时直接重发，收件人就可能收到两封。所以状态变成 unknown 以后，先查权威的发送记录，再决定怎么办。



刚发的邮件查不到，也可能只是记录尚未同步（eventual consistency）。这时应等待可见性窗口结束，再继续查询；只有确认原请求已经终止且没有提交，才能排除它稍后自行完成。如果重试依赖接口的幂等（idempotency）保证，就必须沿用同一个请求标识和同一组参数，而且仍在去重有效期内。Message-ID 能帮助找到同一封邮件，但它本身不负责去重。

参数错了要先改，但改好参数并不能证明上次没执行，仍要核对。权限拒绝则不是多试几次、换一条路径就能解决的。恢复任务时，保留原请求标识、输入版本和核验结果，才知道接下来是在继续同一个请求，还是内容已经变了、需要重新批准。

3.3 模型能力提高后，哪些向导步骤可以删除？

先删去已经不需要的操作指导，每次只删一步；确认风险没有超出容差、用户确实少做了工作，再保留这项简化。

即使模型能力提高十倍，会操作软件，也不等于知道用户愿意接受哪种业务取舍，更不等于已经获准发送文件。教它怎样点击的步骤，可以重新测试后再决定去留；确认意图和授权的步骤，不能因为模型变强就取消。

OSWorld 这类基准的成绩提高了，说明操作能力变强，权限边界并不会跟着改变。它的任务设计可以借用，真实账号、权限和中断恢复仍要另外验收。

测试要用没参与调整的任务，比较危险副作用、选错对象、重复执行和中断恢复，同时统计用户介入了多少次。可接受的风险上限和所需样本量事先定好；不能因为少量试验没有出事故，就撤除整套向导。

来源

01	OpenAI — Computer use https://developers.openai.com/api/docs/guides/tools-computer-use	OpenAI · 官方文档
敏感环节应在执行前确认		
02	OpenAI — Guardrails and human review https://developers.openai.com/api/docs/guides/agents/guardrails-approvals	OpenAI · 官方文档
执行前检查对象、参数与授权窗口		
03	Anthropic — How we contain Claude across products https://www.anthropic.com/engineering/how-we-contain-claude	Anthropic · 官方工程文章
按访问范围与凭证边界判断风险		
04	AWS Builders' Library — Making retries safe with idempotent APIs https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/	Amazon Web Services · 官方工程文章
同一请求标识才能安全重试		
05	RFC 5322 — Internet Message Format https://www.rfc-editor.org/rfc/rfc5322#section-3.6.4	IETF · 技术标准
Message-ID 不提供去重保证		
06	OSWorld 2.0 — Benchmarking computer-use agents on long-horizon real-world tasks https://osworld-v2.xlang.ai/	OSWorld · 研究基准
任务设计可借用，权限须另行验收		
07	OpenAI — Evaluation best practices https://developers.openai.com/api/docs/guides/evaluation-best-practices	OpenAI · 官方文档
生产验收需保留真实任务回归		

类型受约束的输出，怎样才能用于业务决策？

答案符合 schema，仍然可能选错。要让系统自动放行，还需要检查概率分布、在自己的业务上校准，并通过独立验收。

模型只能在几个合法选项里选，这解决的是格式和取值范围；选错了其中一项，业务判断仍然是错的。Jev 的 confidence 反映的是分布有多集中，并不直接等于答对的概率。所以“输出空间锁死”只保证了类型合法。选择是否正确、概率是否经过校准、重复调用会不会跨过动作边界、能否抵抗误导性输入，这些都要分开检查；还要清楚漏掉一个严重问题要付出什么代价。

4.1 严重度得分为 1.99，人工闸门设在 2.00，规则应该怎样写？

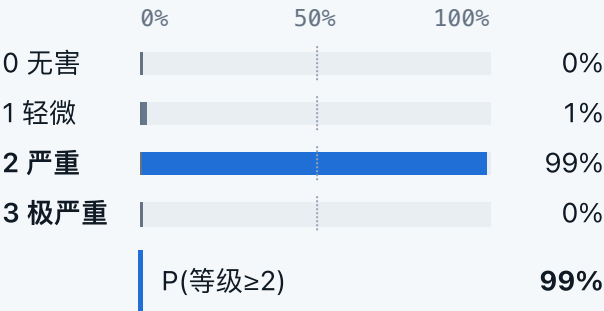
规则不按均值写，按严重等级的概率写：“严重及以上”的概率不超过放行上限（用业务数据校准）才自动放行，超过阻断线就阻断，落在两者之间的送审。

四个等级分别记 0、1、2、3 分，构造下面两组概率。两组均值都低于 2，“低于 2 放行”都会放行，但两组在各等级上的概率差异很大。

图 4.1
同一条“低于2放行”规则下的两组四等级风险分布

构造分布 A（确定性合成示例）

概率，0–100%



均值，按 0–3 计分；竖线为 2



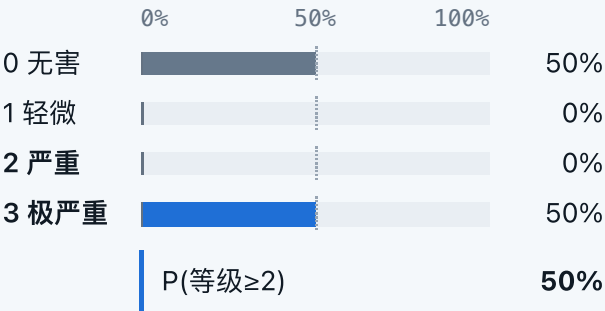
概率向量 [0, 0.01, 0.99, 0]

均值 1.99

“低于 2 放行” 放行

构造分布 B（确定性合成示例）

概率，0–100%



均值，按 0–3 计分；竖线为 2



概率向量 [0.5, 0, 0, 0.5]

均值 1.50

“低于 2 放行” 放行

原始值

分布	P(0 无害)	P(1 轻微)	P(2 严重)	P(3 极严重)	均值	P(等级≥2)	“低于 2 放行”
构造分布 A	0	0.01	0.99	0	1.99	99%	放行
构造分布 B	0.5	0	0	0.5	1.50	50%	放行

分布 A 里“严重”占了 99%，均值却只有 1.99。把 1.99 取整成 2，看似修补了这个例子，却拦不住分布 B：无害和极严重各占一半，均值只有 1.5。均值把整个分布压成一个数，严重尾部也跟着消失，调整小数位也无法补回。

所以规则直接读完整的概率向量，按下面的顺序判断：

条件	动作
输入不完整、模型或 rubric 版本未经验收、输入超出校准范围	送审
P(等级≥2) 不高于放行上限	自动放行
P(等级≥2) 高于阻断线	阻断

介于放行上限与阻断线之间

送审

放行上限在独立的业务测试集上校准，让严重漏报率的置信上界不超过业务能承受的预算；阻断线按误拦的代价来定。已经送审的事项，没有新证据，就不能因为重试一次得到更低分数又改回放行。

4.2 上线前要补哪一份材料？

补一份独立的业务验收与发布批准报告：用自己的业务样本和独立标注，检查字段、中间量和最终决定，写清严重漏报、覆盖率、转人工比例和国内网络下的故障表现，最后由负责人批准。

厂商题集上的校准数字，只能说明模型在那批题上表现如何；换成自己的文件和业务要求，就要重新取样、重新标注。样本要覆盖中文、否定句、日期和数字、长材料、选项顺序、无关信息和对抗内容；同一客户、同一份合同的片段，不能既出现在调参材料里，又出现在测试材料里。

图 4.2
发布报告的内容，以及只核对最终标签时漏检的错误

发布报告内容

标准答案	由独立标注者给出，意见不一致时另行裁决
测试材料	调参、校准和最终测试各用一批，互不重复
报告指标	严重漏报及其置信区间、自动处理覆盖率、转人工比例和成本
网络与故障	国内网络下的延迟、超时率和降级演练结果
用途与批准	批准的用途与禁用用途、模型和 rubric 版本、负责人

只核对最终标签时漏检的错误



只核对最后的通过标签也不够。模型可能取错了字段，却碰巧得出同样的标签；如果这个错误字段后面还要参与计算，这次“答对”反而掩盖了问题。所以验收要查那些后面还会用到的字段和中间量，不能只报一个最终准确率。

同一个模型，拿来监控趋势和拿来放行生产动作，验收标准不一样。报告批准的是某个具体用途；只在监控里验证过的评估器，不能直接拿去当发布闸门。

4.3 同一道题三次结果不同，应该检查什么？

先确认三次的输入和评分条件是否一致，再看差异出在原始数值、标签，还是最终动作上。

输入字节、实际模型版本、rubric、选项顺序、包装层，任何一项改变，三次调用就不能直接比较。排除这些以后，还要看波动造成了什么后果：小数点后变了一点但动作没变，和一次放行、一次送审，性质完全不同。包装层如果把 1.99 转成了整数 2，还要查清业务代码实际读的是哪个字段。

这些信息要和重复实验一起保存，不能最后只剩一个“一致率”，更不能一直重试到某一次通过为止。同一条轨迹重复运行很多次，能检验稳定性，但并没有增加新的业务情境；严重漏报的上界，仍要靠独立的业务测试来估计。

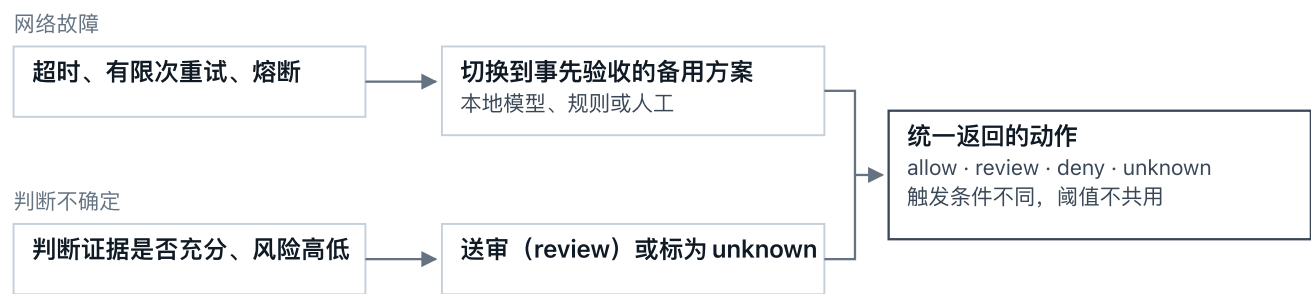
输入完整、模型版本已验收、结果仍在校准范围内，三条都满足，自动判断才有依据。把请求全部交给人工，自动错误可能少了，自动处理的工作却没有完成，所以覆盖率要一起报告。

4.4 服务不稳定和判断不确定，能用同一种降级吗？

服务无法连接，按故障流程恢复或切换；证据不足，就补证或送审。两种情况最后可以返回同一类动作，但触发条件不能共用一套。

网络恢复了，只说明又能收到响应，不说明答案可靠。备用模型能返回分数，也不代表原模型的阈值可以直接套上去。决定下一步之前，先确定当前缺少的是服务响应，还是下判断的依据。

图 4.3
网络故障与判断不确定分别处理



数据去向	接入时确定，不在故障后临时更换供应商或传输区域
高影响动作	校验服务不可用时，只保存草稿或交人工处理

来源

01	TypeSafe — Score https://docs.typesafe.ai/primitives/score.md 分数为等级概率加权，可落在两级之间	TypeSafe · 官方文档
02	TypeSafe — Confidence https://docs.typesafe.ai/confidence.md confidence 只反映分布集中度	TypeSafe · 官方文档
03	TypeSafe — Jev 1.13 jaggedness https://docs.typesafe.ai/model-jaggedness/jev-1.13.md 数字、日期与对抗内容需专项覆盖	TypeSafe · 官方文档
04	TypeSafe — Self-consistency: choices https://docs.typesafe.ai/cookbooks/consistency_choice_cookbook.md 重复一致率不等于结果正确	TypeSafe · 官方文档
05	Pydantic AI — TypeSafe (Jev) https://pydantic.dev/docs/ai/models/typesafe/ 取整等级需核对原始分数字段	Pydantic · 官方文档
06	Pydantic AI — models.typesafe API https://pydantic.dev/docs/ai/api/models/typesafe/ latest 别名随发布变动，需记录实际版本	Pydantic · 官方文档

律师的一次修改，怎样才能成为平台规则？

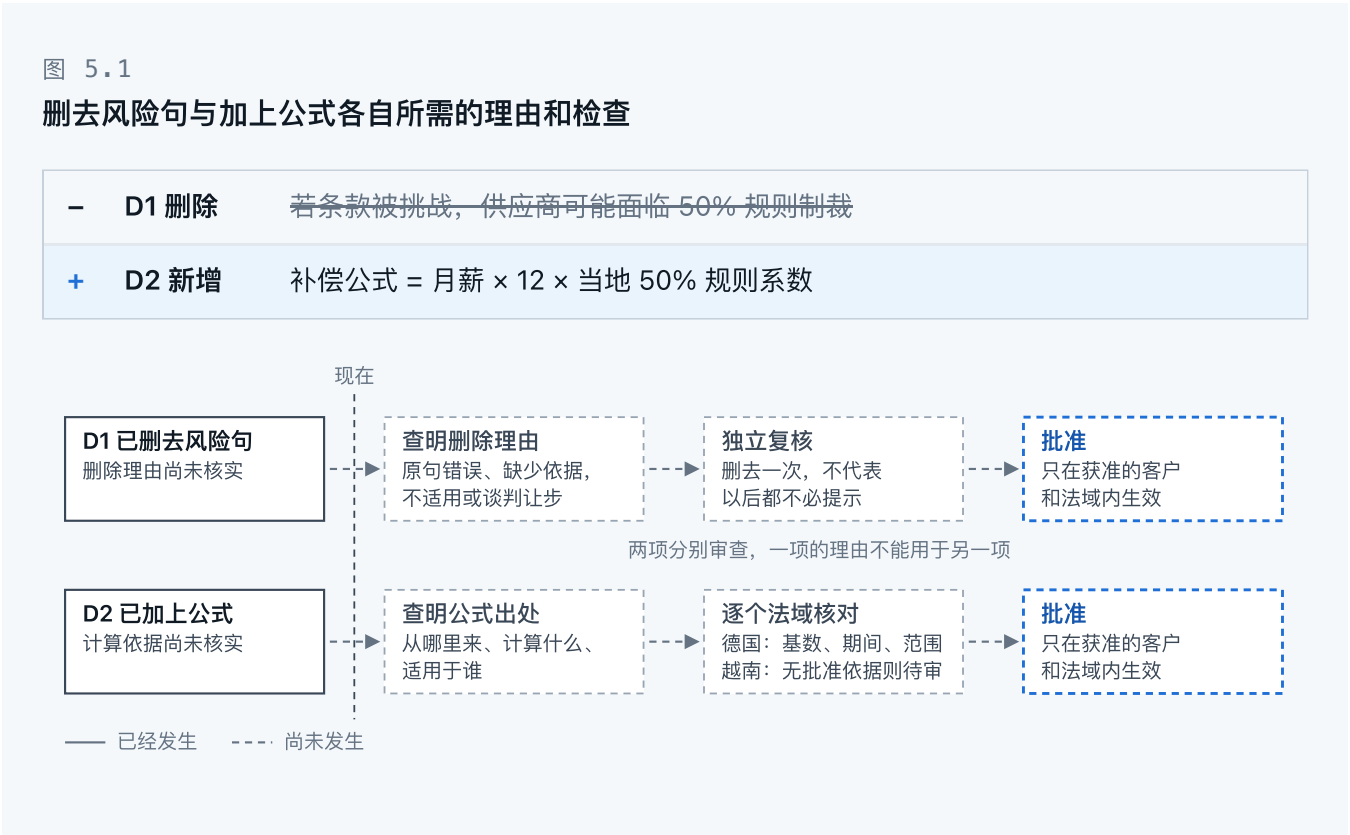
两处修改都先保存下来，但不直接写进全平台生效的 **playbook**。删句和加公式各自查清理由和依据，批准后才只在限定的客户和法域内使用；下周的德国和越南事项，只用各自已经批准的规则。

香港律师替客户 A 修改新加坡就业合同，删了一句风险提示，加了一条补偿公式。律师交付合同，完成的是客户 A 的委托；平台要不要采纳其中的做法，是另一个决定。平台如果只记下“律师改过这份合同”，下次就判断不了该沿用哪一处。经验要能复用，还要知道律师为什么这样改。

5.1 为什么删除一句话和新增一个公式要分开审查？

这次删了风险提示，不代表以后都不用提示；这次加了公式，也不代表其他合同都可以照此计算。两处修改的理由要分别核实。

律师删那句话，可能是原句没有依据，也可能只是它不适用于客户 A；加公式又可能出于另一番考虑。两处虽然是一起改的，接受其中一处的理由，却不能用到另一处。



被删句子里的“50% 规则制裁”和公式里的“50% 规则系数”，要各自查清指的是什么。数字一样，不能说明出自同一条法律规定；客户 A 愿意接受这样的约定，也不能说明别的客户都该这样写。

放到德国合同里，就算碰巧算出一样的金额，算法也还要核对。德国相关规定中的 50% 是竞业限制补偿的下限，基数涉及最后取得的合同给付，浮动部分另有平均规则，计算期间也要和限制期对应。只拿月薪乘 12 再乘 50%，也许能与某个样例的结果相符，这些条件却都被遗漏了。换到越南，没有取得并批准当地依据之前，不能只改国家名称、填一个系数。

5.2 机制要保留什么，修改才能成为规则？

要保留原稿、两处改动各自的理由和依据、作者和委托关系、审查与批准记录，以及父版本和适用范围。做法批准之后才写进带版本的 **playbook**，只给一个新版本号是不够的。

平台先要学会的，是以后怎样检查合同：哪些句子要查法源，公式要核对哪些条件，碰到什么情况需要复核。这些做法存进带版本的 **playbook**，后面的合同就能用上，不必先训练模型权重。要说清一次更新为什么值得采用，就要保留 D1、D2 各自的原稿、理由、反对意见和审查过程；这些历史记录受权限约束，也不会自动进入以后的回答。

图 5.2
保留修改理由和批准过程，再限定可适用的合同范围

修改内容	原稿、逐项 diff 和父版本
修改理由	作者身份、执业法域、修改理由、法源与反对意见
检查与批准	测试结果、审批人和生效时间

四类信息分开保存，读取和使用时都核对范围

合同事实	客户偏好可以影响起草选择，不能覆盖强制规定或平台的权限要求	法律要求	平台核查步骤
------	-------------------------------	------	--------

能不能复用，还要看资料可以给谁用。客户 A 的红线不能拿去回答客户 B 的问题，也不能进入给 B 准备的摘要、记忆或评估材料；到最后才把客户名删掉，之前已经用过 A 资料的影响无法消除。所以读材料、用规则的时候，都要核对客户和适用范围。

客户偏好、合同事实、法律要求和平台流程，也不能混在一起。A 偏好的写法可以用在替 A 起草的合同里，但不能覆盖强制规定，也不能改变平台的权限要求。按 A 的要求交付了一份合同，不等于以后所有客户都要照这样写。

5.3 独立评估怎样做？

原版、只删、只加、两处都改，四个版本分开测试；法源交给熟悉当地法律的人复核，验收材料、评分标准和发布权限都不交给提出修改的一方。

只测改完的整份合同，就算结果更好，也分不清作用来自删句、加公式，还是两者一起。四组对照能把这件事拆开；再换客户、换法域、换作者，看平台会不会在不该沿用的时候照搬。

图 5.3
先比较四个版本，再检查更换客户、法域或作者后的行为

分别测试删除、新增及两者组合

比较对象	待回答的问题	验收条件	状态
原版	—	作为父版本与对照基线	父版本
仅删除 D1（风险句）	原句错误、缺少依据、不适用于这份合同，还是谈判取舍	不据此推定以后都不必提示这类风险	已提出，待核查
仅新增 D2（公式）	公式从哪里来、计算什么、适用于谁	各法域分别取得依据；没有已批准依据时保持待审	已提出，待核查
D1 + D2 组合	改进来自哪一处修改	版本冻结后，用未参与调参的材料验收	D1、D2 分别待审

改变条件后的预期行为

变化	预期行为	验收条件
客户 A 换成客户 B	A 的红线不进入 B 的材料	检查检索、摘要、记忆和评估材料，不只删除最后的客户名
换成另一法域（德国或越南）	不通过替换国家名称补出系数	各法域分别取得依据，由对应法域的复核者核查
德国样例算出相同金额	不据此认定公式完整、可用于其他法域	检查基数、期间和适用范围
越南没有已批准的依据	保持待审	不输出系数
只有尚未生效的版本号	不绕过验收与批准	以规则是否已批准生效为准，不以版本号为准
作者换成知名合伙人	不绕过验收与批准	相同证据得到相同处理

法源和适用条件，由熟悉当地法律的复核者来查。提出修改的学习器，不能自己改验收题、评分标准或发布权限，也不能修改客户隔离要求。版本定下来之后，再用没参与调参的材料验收；如果反复依据同一个隐藏集的结果挑选版本，实际上也是在针对这套题调参。作者再有名、版本号再新，这些检查都省不了。

采用新版时，父版本和依据都保留。后来发现它把原先答对的题答错了，就停用受影响的规则，退回到适用的已批准版本，再根据失败记录提出新的修改。这样下一轮仍能继续学习，已经发现的问题也不会继续影响合同。

5.4 奖励黑客风险在哪里？

如果只奖励报告干净、改动少或测试分高，系统就可能学会删警告、套公式、避开难题，或者把全部请求转交人工。法源、范围、隔离、批准这几条要先满足，效率再高也不能折抵；满足之后，再比较质量和成本。要说系统更善于改进自己，还需另做一层对照。

删掉警告，报告更干净；套上公式，结论显得更确定；跳过难题，正确率更高；记住客户 A 的答案，演示更好看；全部转人工，自动错误更少。每一样都能让某个指标变好，问题却留给了客户或审阅者。所以法源、适用范围、客户隔离、必检项和批准是门槛，要先通过，效率上的收益抵不掉其中任何一项；通过之后，再把质量、覆盖率、人工时间和成本放在一起比较。

这些都做到了，也只回答了一个问题：新版本是否更善于处理合同。要证明平台更善于改进自己，得另做一次比较。

待证明的结论	比较方法	可以支持的判断
新规则更善于完成任务	原版与新版处理同一批未见过的合同，比较错误、覆盖率、审阅时间和总成本	这次规则更新有效
新系统更善于产生下一次改进	使用相同规则版本、失败案例和预算，分别检查新旧学习器提出的修改是否有效；未被采用的尝试也计入成本	在这些条件下，新版更善于提出并验证有效改进

前一项做得好，替代不了后一项的证据；后一项也要跨多轮、跨任务复核，才能排除碰巧成功。RSI 研究里的提出、验证、选择和回滚，可以用来安排这些实验，但证明不了哪条法律公式是对的。平台当前就能使用的，是这次学到的检查方法：删和加分开审，先查清规则从哪来，再交给独立的人验收。至于那条公式，依据和批准仍需另行取得。

来源

01	Google Research — RRSI: Regularized Recursive Self-Improvement of Agent Harnesses https://github.com/google-research/rrsi 候选保留变更历史，评估后再选用	Google Research · 研究代码仓库
02	RSI-Exam — Benchmarking Recursive Self-Improvement through Executable Research https://rsi-exam.ai/ 开发与隐藏评测分离，冻结后验收	RSI-Exam project · 研究基准
03	Weco AIDE ² research report https://www.weco.ai/blog/first-evidence-of-recursive-self-improvement 任务优化不等于改进能力提升	Weco AI · 研究论文或预印本

04	Anthropic — Natural emergent misalignment from reward hacking https://www.anthropic.com/research/emergent-misalignment-reward-hacking 奖励黑客可能引发 misalignment	Anthropic · 研究论文或预印本
05	Singapore Court of Appeal — Man Financial v Wong Bark Chuan David [2007] SGCA 53 https://www.elitigation.sg/gd/s/2007_SGCA_53 竞业限制须辨别可保护利益	Singapore Judiciary eLitigation · 判决
06	Germany — Handelsgesetzbuch §74 https://www.gesetze-im-internet.de/hgb/__74.html 补偿下限为最后合同给付的一半	Gesetze im Internet · 法律条文
07	Germany — Handelsgesetzbuch §74a https://www.gesetze-im-internet.de/hgb/__74a.html 限制期不得超过离职后两年	Gesetze im Internet · 法律条文
08	Germany — Handelsgesetzbuch §74b https://www.gesetze-im-internet.de/hgb/__74b.html 浮动给付按近三年平均计算	Gesetze im Internet · 法律条文
09	Germany — Gewerbeordnung §110 https://www.gesetze-im-internet.de/gewo/__110.html 雇佣关系相应适用 HGB 竞业条款	Gesetze im Internet · 法律条文
10	US Treasury OFAC — Entities Owned by Blocked Persons (50 Percent Rule) https://ofac.treasury.gov/faqs/topic/1521 50% 所有权规则不是补偿系数	U.S. Department of the Treasury, OFAC · 监管机关说明

访问与核对日期：2026-09-23。法律条文和判决仅用于辨认题中出现的概念（如竞业限制补偿、可保护利益、50% 规则的不同含义），不构成法律意见，也不覆盖新加坡、德国以外法域或本题合同的完整适用判断。